Smart Thermal System for Patient Safety Monitoring

Full-Corpus Training, ONNX Export, and Quantization Report

Covers 5 detector(s) with at least one evaluated variant at generation time. Missing variants are noted per section rather than silently omitted.

Dataset Composition

Dataset Notes

Working notes on the three data sources under `data/`, compiled while building the data-infrastructure layer in `Smart-Thermal-System-for-Patient-Safety-Monitoring-dev/thermal_algorithms/training/` (`hdf5_source.py`, `label_join.py`, `multi_source.py`, `split.py`, `pseudo_labels.py`). Everything below was verified against the real files on disk, not assumed from folder names or configs — see each section for how.

The data sources — `room-1` is excluded

`room-1` **is excluded from the dataset entirely** (project owner's decision) — both its recording dates turned out to be too compromised to use, for two independent reasons (see "Why room-1 is excluded" below). Only `waveshare_work` and `synth_room_1-5` are in active use.

	`waveshare_work`	`synth_room_1` … `synth_room_5`
Origin	Manually annotated (project owner)	Synthetic, from a colleague's simulator
Raw format	`ch{0,1,2}_raw_data.npz` + `ch{N}_frames/*.png` + `.mp4`	Chunked `.h5` files, `<date>/cam_{0,1,2}/*.h5`
Labels	Per-frame YOLO `.txt` (fire=class 0, person=class 1) + per-frame `contact_labels.csv`	Room-level event intervals, `labels.csv`
Resolution	80×62 (`Waveshare_26984` profile)	80×62, matches `Waveshare_26984`
fps	8.0	24.0 (confirmed exact)
Sessions	17 scenario folders	5 rooms × 1-3 dates each
Bounding boxes	**Complete** — all 17 scenarios, all 8259 frame-slots (see below)	**None, ever** — only room-level intervals

Class taxonomy → fire/human/contact

`synth_room_*`/`room-1` label events with a single string that bundles several concepts (e.g. `Contact_2+Humans+Fire`). Per-problem booleans are derived by substring match (`thermal_algorithms/training/label_join.py:classify_event`):

```
fire: "Fire" in class
human: class != "Empty" and class != "Fire" (bare fire alone has no human)
contact: "Contact" in class or "Violence" in class # Violence folded into contact
```

`*+fall` classes (patient-fall scenario) are **out of scope** — no fall detector exists anywhere in the codebase. Fall frames still count correctly as human-positive (they're not "Empty"); the fall-specific signal is simply not tracked separately.

Real hour totals across all 6 `labels.csv` files (synth_room_1-5 + room-1, 123 labeled hours):

Problem	Positive	Negative	Ratio
Fire	~37.0h (30.1%)	~86.0h (69.9%)	1:2.3
Human / restricted-area	~91.5h (74.4%)	~31.5h (25.6%)	2.9:1
Contact (incl. Violence)	~3.85h (3.1%)	~119.15h (96.9%)	1:31

These hour totals include room-1 (recorded before its exclusion) and are kept here as a historical reference for the taxonomy/ratio methodology; they are no longer part of the active training-data accounting now that room-1 is excluded.

Waveshare's real per-frame labels, now fully restored (checked directly via `DatasetIndex`/`FrameLevelDataset`/`FireFrameDataset`/`ContactFrameDataset`, not estimated), across all 17 scenarios / 8259 frame-slots:

Problem	Positive	Negative	Ratio
Human	7421 (89.9%)	838 (10.1%)	8.9:1
Fire	1373 (16.6%)	6886 (83.4%)	1:5.0
Contact	188 (6.8%)	2565 (93.2%)	1:13.6

The fire count (1373) matches `Weights/manifest.json`'s recorded `fire_positives` from the prior training run exactly — strong confirmation the restored annotations are faithful to the original, pre-loss set. This also resolves the `hog_svm` manifest ratio flagged earlier as looking anomalous (7006/7629 ≈ 91.8% positive) — it isn't an anomaly, this dataset is genuinely this densely populated with people; the 89.9% figure above lines up closely.

Bounding-box availability

waveshare_work's bounding boxes are now complete — restored by the project owner after initially being lost (see the earlier `Weights/`-baseline/pseudo-label workaround below, now superseded for this source). Verified programmatically, not just by file count: all 17 scenarios show `has_labels=True` on all 3 channels, and `FrameLevelDataset`/`FireFrameDataset` enumerate real detections matching the counts above.

No bounding boxes exist anywhere in `synth_room_*` — only room-level event intervals. This is a source characteristic, not a gap to close: it can feed frame-level presence-only training/eval (`FireAlert`, `ContactEvent`, or a plain `(Frame, bool)` for human presence), but never bbox-regression (`MobileNetSSDDetector`) or IoU-based fire evaluation. Now that waveshare has real boxes, IoU-based fire/human evaluation is finally possible — just scoped to waveshare.

Raw sensor encoding

`.h5` chunk `frames` datasets are `uint16`, **centi-Celsius**: `temp_C = raw / 100`. Verified two independent ways: (1) a sample frame's raw values (~2953-3032) map to ~29.5-30.3°C, a physically plausible room temperature; (2) the synthetic simulator's own `config.json` clips its preview-video rendering to `thermal_clip_c: [15.0, 45.0]`, which the same `/100` conversion falls comfortably inside.

Why `room-1` is excluded

`room-1` had two recording dates, and each turned out to be unusable for a different reason — investigated in full before deciding to exclude the source entirely (project owner's decision).

`2026-06-23`: 2 of 13 `.h5` chunks in `cam_1` fail to open at all (`OSError: wrong B-tree signature` — genuine HDF5-level corruption, not a format quirk), covering roughly a 20-minute window. Every other file on this date, across all 3 cameras, reads back completely clean pixel data (0% frozen — see below).

`2026-06-30`: every file opens fine, but the actual pixel content is badly degraded — checked directly against raw `.h5` bytes, bypassing all reader/cache code, per camera:

Camera	Frozen frames (std < 1e-4)
cam_0	18.6%
cam_1	68.6%
cam_2	**100% — every single frame, the entire ~3-hour session, is the exact constant value 27.31°C**

This was found indirectly: `GlobalNormPreprocessor` crashed `FireSVMDetector` with a NaN feature vector on this data (see `fire_svm.py`'s NaN-handling fix below), and tracing why led to the frozen frames. **This reverses an earlier (wrong) conclusion**: `2026-06-30` was initially treated as "the clean date" and `2026-06-23` excluded for its 2 corrupted files — backwards, since `2026-06-23` is actually far cleaner overall. Since neither date is both complete *and* clean, and 3-view contact detection needs all cameras trustworthy at once, `room-1` is excluded from the active dataset rather than patched around.

The technical work already done to read `room-1` correctly (per-session fps inference, chunk-chaining for broken timestamps — both in `hdf5_source.py`) stays in the codebase; it generalizes to any future

real-hardware source with the same acquisition-pipeline quirks, even though it's not driving any current training.

synth_room_3/4/5's labels.csv use a different internal Room_ID than their folder name

Confirmed 2026-08-07 (project owner) while debugging a `ValueError: no label intervals given` crash partway through a full-corpus training run: three of the five `synth_room_*` folders' `labels.csv` files were copied from a differently-numbered room in the original generation set and never relabeled to match their containing folder's name.

Folder	`Room_ID` values actually found in its own `labels.csv`
`synth_room_1`	`synth_room_1` (matches)
`synth_room_2`	`synth_room_2` (matches)
`synth_room_3`	`synth_room_4`
`synth_room_4`	`synth_room_6`
`synth_room_5`	`synth_room_8`

Each mismatched file is internally consistent (every row across both its dates uses the same wrong ID) and the gaps aren't a simple constant offset (+1, +2, +3) — consistent with these three folders having been cherry-picked from a larger originally-generated pool and renamed on disk without touching the label file inside. The `.h5` pixel data and its own folder's `labels.csv` are treated as a correctly-matched pair (per the project owner) — it's the folder name that disagrees with the label file's internal bookkeeping, not pixel data mismatched with the wrong labels.

Fix: `thermal_algorithms/training/label_join.py:detect_room_id` scans a `labels.csv` for the `Room_ID` it actually uses on a given date, and `HDF5Session.__init__` (`thermal_algorithms/training/multi_source.py`) filters by that detected value instead of the folder-derived name. `HDF5SessionRef.room_id` (folder-derived) is still used for `.scene`/display naming, since that's just a label for humans, not a filter key. `detect_room_id` raises loudly if a file ever has more than one distinct `Room_ID` for one date — that would be a different, more serious problem than the confirmed case above and needs human review, not an automatic pick.

Baseline weights and pseudo-labels (superseded for waveshare, kept as infrastructure)

`Weights/` (project root, sibling to `data/`) holds a prior full training run on `waveshare_work`, from before its bbox/contact annotations were lost: `fire_svm.pkl`, `hog_svm.pkl`, `mobilenet_ssd.pt`, `mv_stgcn.pt`, `thermo_x3d.pt`. Imported into `checkpoints_baseline_waveshare/` via `scripts/import_baseline_weights.py` and round-trip verified against real waveshare frames (load → predict → sane output, no crashes). Known limitation: `mv_stgcn.pt` has no embedded

homography, so the imported `MVSTGCNDetector` produces zero actors until a real homography is supplied separately.

`scripts/generate_pseudo_labels.py` used the imported `MobileNetSSDDetector` to regenerate approximate person boxes for the (then-)16 unannotated waveshare scenarios. **Superseded**: the project owner has since restored the real annotations for all 17 scenarios (see "Bounding-box availability" above), so this pseudo-label run and its output (`data_artifacts/waveshare_pseudo_person_labels.json`) are no longer needed and should not be used — real ground truth exists everywhere now. The mechanism itself (`thermal_algorithms/training/pseudo_labels.py`) is kept as tested, reusable infrastructure in case a future data source needs the same treatment (e.g. `synth_room_*` never has bboxes and could, in principle, be pseudo-labeled the same way, though no such run has been done).

Session-level train/test split

Real sources are split by whole session, not by frame, to avoid leakage between near-duplicate consecutive frames (`thermal_algorithms/training/split.py`). Verified against real waveshare data: 17 sessions → 14 train / 3 test (seeded, deterministic). Synthetic data is used wholesale for training per the project's task-3 instruction, not run through this split. `room-1` is excluded (see above), so it's not a candidate for splitting.

Performance note: SessionMetadata frame loading is now cached

`SessionMetadata.load_frame()`/`load_frames()` used to re-open and fully re-decompress the whole npz archive on every single call — negligible for a few calls, but it made iterating a real session (hundreds to thousands of frames from the same file) impractical: a full `FrameLevelDataset`/`FireFrameDataset`/`ContactFrameDataset` pass over all of `waveshare_work` (8259 frame-slots) timed out at 2+ minutes before this was fixed. Now cached per npz path (`thermal_algorithms/training/datasets.py:_load_npz_frames_cached`, `functools.lru_cache`) — the same full-dataset pass now takes ~2.5 seconds. Safe to cache: these files are never modified in place during a run, and every caller immediately `.astype()`s the result into a fresh array rather than mutating it.

Mathematical Derivations of the Core Detection Algorithms

Smart Thermal System for Patient Safety Monitoring — Beer Sheva Mental Health Center

This section derives, from first principles, the seven detection algorithms implemented in `thermal_algorithms/`. Each subsection matches the code actually shipped in the repository — file paths and parameter names are cited so the mathematics can be checked line-by-line against the implementation, rather than against a generic textbook description of the method. Where an existing evaluation report (`reports/*.tex`) already worked out part of a derivation, that content is adapted and cited rather than re-derived from scratch.

---

1. Otsu Thresholding — `fire_detection/otsu_utils.py::otsu_segment`

1.1 Formulation

Otsu's method converts a grayscale (here: temperature) image into a binary foreground/background mask by choosing the single intensity threshold `k^\*` that best separates the pixel population into two classes, using only the image's own intensity histogram — no external calibration, no labeled data.

Given a float32 thermal frame `data` with per-frame minimum `d_{\min}` and maximum `d_{\max}`, the implementation first rescales every pixel into `n\_bins = 256` integer bin indices:

$$\text{norm}(x) = \frac{x - d_{\min}}{d_{\max} - d_{\min} + \varepsilon}\,(n_{\text{bins}}-1), \qquad \varepsilon = 10^{-6}$$

and builds a histogram $h_0, h_1, \dots, h_{n_{\text{bins}}-1}$ of bin counts over the frame. Normalizing by the total pixel count $N = \sum_k h_k$ gives the empirical probability mass function

$$p_k = \frac{h_k}{N}, \qquad k = 0,\dots,n_{\text{bins}}-1, \qquad \sum_k p_k = 1 .$$

1.2 Derivation: between-class variance `\sigma_B^2(k)`

For a candidate threshold `k`, every pixel is assigned to class $C_0=\{0,\dots,k\}$ ("background") or $C_1=\{k+1,\dots,n_{\text{bins}}-1\}$ ("foreground"). Define the class occupation probabilities

$$\omega_0(k) = \sum_{i=0}^{k} p_i, \qquad \omega_1(k) = \sum_{i=k+1}^{n_{\text{bins}}-1} p_i = 1-\omega_0(k),$$

and the class means

$$\mu_0(k) = \frac{1}{\omega_0(k)}\sum_{i=0}^{k} i\,p_i, \qquad \mu_1(k) = \frac{1}{\omega_1(k)}\sum_{i=k+1}^{n_{\text{bins}}-1} i\,p_i \ .$$

The code computes these via running cumulative sums (`w0 = cumsum(p)`, `mu_k = cumsum(idx * p)`), which is the numerically efficient form:

$$\omega_0(k) = \sum_{i\le k} p_i, \qquad \underbrace{\sum_{i\le k} i\,p_i}_{\texttt{mu\_k}} = \omega_0(k)\,\mu_0(k).$$

The total (frame-wide) mean is `\mu_T = \sum_i i\,p_i = \texttt{mu\_k}[-1]`, and it decomposes as a weighted average of the class means for *any* threshold `k`:

$$\mu_T = \omega_0(k)\,\mu_0(k) + \omega_1(k)\,\mu_1(k). \tag{1}$$

Total variance decomposition. The total intensity variance of the histogram can always be split into a *within-class* term and a *between-class* term:

$$\sigma_T^2 = \underbrace{\omega_0\sigma_0^2 + \omega_1\sigma_1^2}_{\sigma_W^2(k)\ \text{(within-class)}} + \underbrace{\omega_0(\mu_0-\mu_T)^2 + \omega_1(\mu_1-\mu_T)^2}_{\sigma_B^2(k)\ \text{(between-class)}}. \tag{2}$$

Since `\sigma_T^2` does not depend on `k` (it is a property of the whole histogram, fixed once the frame is fixed), **maximizing `\sigma_B^2(k)` over `k` is exactly equivalent to minimizing `\sigma_W^2(k)`** — Otsu's criterion simultaneously makes the two classes as internally homogeneous as possible *and* as mutually separated as possible, because those are the same objective viewed from opposite sides of Eq. (2).

Closed form used in the code. Substituting `\mu_1 = (\mu_T - \omega_0\mu_0)/\omega_1` from Eq. (1) into the between-class term of Eq. (2) and simplifying (a standard algebraic reduction — see e.g. Otsu, 1979) collapses the two-term expression into a single ratio that needs only `\omega_0(k)` and the cumulative first moment:

$$\sigma_B^2(k) = \omega_0(k)\,\omega_1(k)\,\bigl(\mu_1(k)-\mu_0(k)\bigr)^2 = \frac{\bigl(\mu_T\,\omega_0(k) - \texttt{mu\_k}(k)\bigr)^2}{\omega_0(k)\,\omega_1(k)}. \tag{3}$$

This is precisely the `sigma_b2` array computed in the code:

```
sigma_b2 = (mu_t * w0 - mu_k) ** 2 / (w0 * w1) # guarded where w0>0 and w1>0
```

Equation (3) is a function of `k` alone (all other quantities are frame-wide constants), so it can be evaluated for every one of the 256 candidate thresholds in `O(n_{\text{bins}})` time given the cumulative sums — no per-threshold pass over the pixels is needed. The optimal threshold is

$$k^* = \arg\max_k \ \sigma_B^2(k), \qquad \texttt{threshold} = d_{\min} + \frac{k^*}{n_{\text{bins}}-1}(d_{\max}-d_{\min}),$$

exactly `k_star = argmax(sigma_b2)` followed by the de-normalization back to physical temperature units. The binary mask is then `data >= threshold`.

1.3 Why this method is theoretically justified here

Otsu's threshold is the **maximum-likelihood boundary under a two-Gaussian mixture model with equal variances and equal priors** projected onto the 1-D intensity axis — i.e., it is the optimal separator when the frame genuinely contains two thermally distinct populations (e.g., "warm scene" vs. "hot blob"). It requires no labeled data and adapts automatically to per-frame dynamic range, which matters here because ambient ward temperature drifts and different fire/ignition scenes have very different absolute temperature scales. This makes it a reasonable *region proposal* stage (as used by `FireSVMDetector`, §2) even though — as `reports/fire_detection_report.tex` documents — a global two-class assumption is fragile when the "hot" class is dominated by warm human bodies rather than by the (much smaller, much hotter) fire blob; that failure mode motivated the hot-pixel bypass fix described there, distinct from the pure derivation given here.

1.4 Morphological post-processing

After thresholding, the code applies

```
eroded = cv2.erode(raw_mask, morph_kernel, iterations=1)
dilated = cv2.dilate(eroded, morph_kernel, iterations=2)
```

Erosion (`A \ominus B = \{z : B_z \subseteq A\}`, i.e. keep a foreground pixel only if the *entire* structuring element `B` centered there is inside the mask) shrinks every foreground blob by one structuring-element radius. A single hot pixel with no hot neighbors — sensor noise, a stray dead-pixel glitch, a one-pixel thermal reflection — fails the "entire kernel inside" test and is deleted outright. This is a **noise-removal** step: it enforces spatial coherence by requiring a candidate region to have some minimum extent, not just isolated hot pixels.

Dilation (`A \oplus B = \{z : B_z \cap A \neq \varnothing\}`, i.e. grow a region wherever the kernel touches any foreground pixel) then grows the surviving blobs back out, applied *twice* so that the net effect on a surviving multi-pixel blob is a **one-kernel-radius net expansion** relative to the original mask (one erosion undone, plus one extra dilation). This **consolidates** nearby fragments of what is physically a single hot object — e.g. a blob that Otsu's hard threshold happened to split into two disconnected pixel clusters due to noise — into one connected component, and slightly pads the region so `extract_blobs`'s bounding box does not clip the true extent of the hot object. Erosion-then-dilation in this order (rather than dilation-then-erosion, i.e. morphological *closing*) is deliberately asymmetric: it guarantees single/isolated-pixel noise is permanently removed (erosion is a strict, one-way filter here) while the surviving structure is *grown*, not just closed — appropriate for a detector whose failure mode of most concern (per §1.3) is a fire blob that is small to begin with.

---

2. FireSVMDetector — `fire_detection/fire_svm.py`

2.1 Formulation

FireSVMDetector is a two-class classifier (SAFE = -1 vs. ACTIVE_COMBUSTION = +1 in the classical SVM sign convention; the code uses sklearn's \{0,1\} label convention internally, which is equivalent) over a **6-dimensional hand-engineered feature vector** computed from the hottest connected component returned by otsu_segment + extract_blobs (§1):

$$x = \bigl(\,T_{\max},\ T_{\text{mean}},\ T_{\text{std}},\ \text{area},\ \text{skew},\ \text{kurt}\,\bigr) \in \mathbb{R}^6 .$$

Skewness and kurtosis are the standardized third and fourth central moments of the blob's pixel temperatures \{t_i\}_{i=1}^n (biased/population form, matching scipy.stats.skew/kurtosis defaults):

$$\text{skew} = \frac{\tfrac{1}{n}\sum_i (t_i-\bar t)^3}{\bigl(\tfrac{1}{n}\sum_i(t_i-\bar t)^2\bigr)^{3/2}}, \qquad \text{kurt} = \frac{\tfrac{1}{n}\sum_i (t_i-\bar t)^4}{\bigl(\tfrac{1}{n}\sum_i(t_i-\bar t)^2\bigr)^{2}} - 3 .$$

These capture *shape* rather than magnitude: a flame's pixel-temperature distribution is asymmetric (positive skew — a hot core trailing to cooler edges) and heavy-tailed/peaked (positive excess kurtosis — flicker produces outlier-hot pixels against a cooler bulk), whereas a warm human torso at a fairly uniform \sim36 °C is close to symmetric and mesokurtic. This is why the model can distinguish "small hot flickering thing" from "large steady warm thing" even when the two happen to share a similar T_{\max} (adapted from reports/checkpoint_report.tex, "Skewness and kurtosis" paragraph, which documents the identical six-feature vector).

Before classification, features are z-scored using StandardScaler fit on the training set: $\hat{x}_j = (x_j-\mu_j)/\sigma_j$.

2.2 The primal soft-margin QP

FireSVMDetector trains an sklearn.svm.SVC — a soft-margin kernel SVM. For labeled training pairs (x_i,y_i), y_i \in \{-1,+1\}, and a feature map \phi(\cdot) (identity for the linear kernel, the implicit RBF map for kernel="rbf", the code's default), the classifier seeks a hyperplane w^\top\phi(x)+b=0 that separates the classes with maximum margin while tolerating some misclassification. Introducing one slack variable \xi_i \ge 0 per training point to allow margin violations, the **primal problem** is the convex quadratic program

$$\min_{w,\,b,\,\xi} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \qquad \text{s.t.} \quad \begin{cases} y_i\bigl(w^\top\phi(x_i)+b\bigr) \ge 1-\xi_i, & i=1,\dots,n\\ \xi_i \ge 0, & i=1,\dots,n . \end{cases} \tag{4}$$

\xi_i=0 means point i is correctly classified and outside the margin corridor; 0<\xi_i<1 means it is correctly classified but inside the corridor; \xi_i>1 means it is misclassified. C (the constructor's svm_C, default 1.0) is the unit price of a margin violation: it is the single knob trading margin width against training-set fit, and is exactly the inverse of an L_2 weight-decay coefficient when Eq. (4) is rewritten in unconstrained hinge-loss form,

$$\min_{w,b} \ \sum_i \max\bigl(0,\ 1-y_i(w^\top\phi(x_i)+b)\bigr) + \frac{1}{2C}\|w\|^2,$$

i.e. **hinge loss plus ℓ_2 regularization**, with $C\to\infty$ recovering the hard-margin SVM and $C\to0$ making all violations free (adapted from `reports/checkpoint_report.tex`, "Why C is a regularizer").

2.3 The Lagrangian and the dual problem

Introduce Lagrange multipliers $\alpha_i \ge 0$ for the margin constraints and $\mu_i \ge 0$ for the slack non-negativity constraints:

$$\mathcal{L}(w,b,\xi,\alpha,\mu) = \frac{1}{2}\|w\|^2 + C\sum_i \xi_i - \sum_i \alpha_i\Bigl[y_i(w^\top\phi(x_i)+b) - 1 + \xi_i\Bigr] - \sum_i \mu_i \xi_i. \tag{5}$$

Setting the gradient of $\mathcal{L}$ with respect to each primal variable to zero (stationarity — see §2.4) gives

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_i \alpha_i y_i \phi(x_i) = 0 \ \Rightarrow\ w = \sum_i \alpha_i y_i \phi(x_i), \tag{6}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_i \alpha_i y_i = 0 \ \Rightarrow\ \sum_i \alpha_i y_i = 0, \tag{7}$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \ \Rightarrow\ \mu_i = C-\alpha_i \ \Rightarrow\ 0 \le \alpha_i \le C. \tag{8}$$

Substituting (6)–(8) back into (5) eliminates w, b, ξ entirely (the hallmark of a QP dual) and leaves a maximization over α alone, in which the data enters *only* through pairwise inner products $\langle\phi(x_i),\phi(x_j)\rangle$:

$$\max_{\alpha}\ \sum_{i=1}^n \alpha_i - \frac{1}{2}\sum_{i,j=1}^n \alpha_i\alpha_j\,y_i y_j\,\langle\phi(x_i),\phi(x_j)\rangle \qquad \text{s.t.}\quad 0 \le \alpha_i \le C,\ \ \sum_i \alpha_i y_i = 0. \tag{9}$$

This is the **dual problem**, adapted from `reports/checkpoint_report.tex` ("The dual and the kernel trick"), and is itself a convex QP (concave objective maximized, equivalently convex minimized, over a box-and-hyperplane feasible set) — solved by sklearn's SVC internally via **SMO** (Sequential Minimal Optimization), which repeatedly picks the pair (α_i,α_j) that most violates optimality and updates it to its closed-form constrained optimum. There is consequently no learning-rate, epoch count, or gradient-descent schedule for this model — training is an exact convex optimization, and the only capacity knobs are C and (for RBF) γ.

2.4 KKT conditions

For a general constrained optimization $\min_x f(x)$ s.t. $g_i(x)\le 0$, $h_j(x)=0$, the **Karush–Kuhn–Tucker (KKT) conditions** at a candidate optimum (x^*,λ^*,ν^*) are:

1. **Stationarity**: \nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0. 2. **Primal feasibility**: g_i(x^*)\le 0, h_j(x^*)=0 for all i,j. 3. **Dual feasibility**: \lambda_i^* \ge 0 for all i. 4. **Complementary slackness**: \lambda_i^* g_i(x^*) = 0 for all i.

Mapping this template onto the soft-margin SVM of Eq. (4), with inequality constraints g_i^{(1)}(w,b,\xi) = 1-\xi_i-y_i(w^\top\phi(x_i)+b) \le 0 and g_i^{(2)}(\xi)=-\xi_i\le 0, and multipliers \alpha_i \ge 0, \mu_i \ge 0 respectively:

- **Stationarity** — exactly Eqs. (6)–(8) above: w=\sum_i\alpha_i y_i\phi(x_i), \sum_i\alpha_i y_i = 0, \mu_i = C-\alpha_i.
- **Primal feasibility** — y_i(w^\top\phi(x_i)+b) \ge 1-\xi_i and \xi_i \ge 0 for every training point.
- **Dual feasibility** — \alpha_i \ge 0 and \mu_i \ge 0 (equivalently, via (8), 0\le\alpha_i\le C).
- **Complementary slackness** — two conditions, one per constraint family:

$$\alpha_i\Bigl[y_i(w^\top\phi(x_i)+b)-1+\xi_i\Bigr] = 0, \qquad \mu_i\,\xi_i = 0 \ . \tag{10}$$

Equation (10) is what gives the SVM its sparse **support-vector** structure and its three-way partition of the training set:

- If \alpha_i = 0: point i contributes nothing to w (Eq. 6) — it lies strictly outside the margin and is *not* a support vector.
- If 0 < \alpha_i < C: then \mu_i = C-\alpha_i > 0, so by the second half of (10) \xi_i=0; combined with the first half of (10), y_i(w^\top\phi(x_i)+b)=1 exactly — the point sits **on** the margin boundary. This is what makes b recoverable in closed form from any such point.
- If \alpha_i = C: then \mu_i=0, and \xi_i is free to be positive — the point lies inside the margin or is misclassified.

In the fitted SVC object, _svm.n_support_/support_vectors_ are exactly the points with \alpha_i>0; reports/checkpoint_report.tex reports 874 of 6,497 training points as support vectors (\sim13%) for this exact model, which is a direct empirical read-out of how many points satisfy \alpha_i>0 under conditions (10).

2.5 Why we are eligible to use KKT here

KKT conditions are, in general, only **necessary** for optimality (for arbitrary smooth constrained problems), and are of little use if strong duality does not hold — a solution to the dual need not then equal the primal optimum, and the multipliers found need not certify anything about the primal solution. The reason the soft-margin SVM training procedure can *rely* on KKT — treat it as a two-way equivalence used both to derive the dual (§2.3) and to certify convergence during SMO — is a specific, checkable chain of facts about problem (4):

1. **The primal is a convex program.** The objective \tfrac12\|w\|^2 + C\sum_i\xi_i is a positive-semidefinite quadratic form in w plus a linear function of \xi — jointly convex in (w,b,\xi).

Every constraint, y_i(w^\top\phi(x_i)+b)\ge 1-\xi_i and \xi_i\ge0, is **affine (linear)** in (w,b,\xi) — hence both the feasible region is convex (an intersection of halfspaces) and the constraint functions are convex, satisfying the hypotheses required for KKT to be more than "necessary-only." 2. **Slater's condition holds trivially.** Slater's constraint qualification requires a point that is *strictly* feasible for every inequality constraint. Any (w,b) (e.g. w=0,b=0) combined with \xi_i chosen large enough — e.g. \xi_i = 2 for every i — satisfies y_i(w^\top\phi(x_i)+b)=0 \ge 1-\xi_i = -1 and \xi_i=2>0 **strictly**, for *every* training set, with no assumption of linear separability. Soft-margin SVM training is feasible by construction (unlike the hard-margin SVM, whose feasible set can be empty for non-separable data) — this is precisely why the slack variables were introduced in the first place, and it is what guarantees Slater's condition holds for this problem regardless of the input data. 3. **Convexity + Slater ■ strong duality.** By convex-optimization theory (Slater's theorem), a convex problem satisfying Slater's condition has **zero duality gap**: the primal optimal value equals the dual optimal value, and — critically — a primal-dual pair (w^*,b^*,\xi^*,\alpha^*,\mu^*) is optimal for both problems **if and only if** it satisfies the KKT conditions. This "if and only if" is exactly the upgrade from "KKT is necessary" to **"KKT is both necessary and sufficient"** that licenses the derivation in §2.3–2.4: we are not merely hoping a KKT point is the optimum, we have a theorem guaranteeing it is. 4. **Consequence for SMO.** Because KKT is sufficient here, SMO's stopping rule — "no training example violates the KKT conditions by more than a numerical tolerance" — is a valid certificate of (near-)global optimality, not a heuristic. This is also why FireSVM training has no notion of local minima, random restarts, or optimizer instability: the feasible set is convex, the objective is convex, and any KKT point is *the* global optimum.

This is the complete eligibility argument requested: convex QP \Rightarrow Slater trivially satisfied by a feasible soft-margin problem with \xi\ge 0 \Rightarrow strong duality \Rightarrow KKT necessary and sufficient.

2.6 The RBF kernel, Mercer's theorem, and the kernel trick

The dual (9) depends on the data only through inner products \langle\phi(x_i),\phi(x_j)\rangle. The **kernel trick** replaces this inner product with a kernel function k(x_i,x_j) computed directly on the raw inputs, without ever constructing \phi:

$$k(x,x') = \exp\bigl(-\gamma\|x-x'\|^2\bigr), \qquad \gamma = \texttt{gamma="scale"} \Rightarrow \gamma = \frac{1}{d\cdot\mathrm{Var}(X)},$$

the RBF (Gaussian) kernel used by FireSVMDetector(kernel="rbf") (its default, though kernel="linear" is also exposed as a constructor option). This substitution is valid — i.e. the resulting optimization is still exactly the dual of *some* legitimate linear-in-feature-space SVM — precisely because of **Mercer's theorem**: any symmetric function k(x,x') that is positive semi-definite (i.e. for any finite set of points \{x_i\}, the Gram matrix K_{ij}=k(x_i,x_j) is PSD) can be written as k(x,x')=\langle\phi(x),\phi(x')\rangle for *some* feature map \phi into a (possibly infinite-dimensional) Hilbert space. The RBF kernel satisfies this PSD requirement (it is a Gaussian, whose Fourier transform — its spectral density — is non-negative everywhere, which is the Bochner-theorem condition for positive-definiteness). Consequently:

- We never need to compute or store `\phi(x)` — every place `\langle\phi(x_i),\phi(x_j)\rangle` appears in training (Eq. 9) or in the decision function (§2.7), it is replaced by `k(x_i,x_j)`, computed in `O(d)` time on the original 6-D features.
- Taylor-expanding `e^{-\gamma\|x-x'\|^2}` shows the implicit feature space contains monomials of *every* degree simultaneously — the RBF-SVM is a universal approximator over the 6-D feature space, able to express nonlinear decision boundaries (e.g. "hot AND small OR moderately-hot AND heavy-tailed") that a linear hyperplane over the raw 6 features cannot, without ever paying for the infinite dimensionality explicitly (adapted from `reports/checkpoint_report.tex`, "The dual and the kernel trick" / "The RBF kernel").

2.7 Decision function and probability calibration

The trained decision function, applied to a new (scaled) feature vector `\hat x`, is

```
f(\hat x) = \sum_{i \in \mathrm{SV}} \alpha_i y_i\, k(x_i,\hat x) + b,
            \qquad \hat y = \operatorname{sign} f(\hat x),
```

a similarity-weighted vote of the stored support vectors — only points with `\alpha_i>0` (§2.4) need be kept, which is why the persisted checkpoint (`_state_dict` pickles the fitted SVC) is compact. Because `probability=True` is passed to SVC in `fit()`, sklearn additionally fits **Platt scaling** — a 1-D logistic regression `P(\text{fire}\mid x) = \sigma(a f(x)+c)` on cross-validated decision values — giving the calibrated `confidence` field returned in every `FireAlert` (`predict_proba` in the code).

2.8 Implementation notes

- Feature order and NaN handling: `_frame_to_feature_vector` uses the *hottest* blob (`max(blobs, key=lambda b: b["max_temp"])`) among all connected components from `otsu_segment+extract_blobs`; a degenerate zero-variance blob produces NaN skew/kurtosis (matching scipy's convention) which is mapped to `0.0` via `np.nan_to_num` before scaling, since the scaler/SVM cannot accept non-finite input.
- `class_weight` is a constructor-time parameter (not a `fit()` argument) because sklearn fixes it at estimator construction; `'balanced'` reweights the dual's box constraint per-class (`\alpha_i \le C_{y_i}` with class-dependent `C`) to counteract the fire-class rarity noted in `data/DATASET_NOTES.md`.
- `resolution_behavior = "invariant"`: because the feature vector is 6 scalar statistics rather than raw pixels, one checkpoint transfers across the MLX90640 and Waveshare sensor profiles without retraining.

---

3. HOGSVMDetector — `human_detection/hog_svm.py`

3.1 Formulation

`HOGSVMDetector` treats human detection as **shape classification over a sliding window**: at every window position (and every scale in a pyramid), a Histogram-of-Oriented-Gradients (HOG) descriptor is computed and scored by a trained **linear** SVM (`sklearn.svm.LinearSVC`); windows scoring above `score_threshold` become candidate detections, and greedy NMS collapses overlapping candidates.

3.2 Derivation: the HOG descriptor

Let $I(x,y)$ denote the (per-window, normalized — see §3.5) pixel intensity. HOG proceeds in three stages, matching `skimage.feature.hog(patch, orientations=9, pixels_per_cell=cell_size, cells_per_block=block_size_cells, block_norm="L2-Hys")` exactly as called in `_hog_features`.

1. Gradient computation. At every pixel, horizontal and vertical gradients are computed (skimage's default is the centered finite difference $[-1,0,1]$):

$$G_x(x,y) = I(x+1,y)-I(x-1,y), \qquad G_y(x,y) = I(x,y+1)-I(x,y-1),$$

from which the gradient magnitude and orientation are

$$m(x,y) = \sqrt{G_x^2+G_y^2}, \qquad \theta(x,y) = \arctan\!\Bigl(\frac{G_y}{G_x}\Bigr) \in [0^\circ,180^\circ) \text{(unsigned)}.$$

Thermal edges (a person silhouette against a cooler/warmer background) are exactly the kind of locally coherent gradient structure this captures — HOG's premise is that object *shape*, encoded in the local distribution of edge orientations, is a more scale/illumination-robust cue than raw intensity, which matters here because ambient ward temperature varies but silhouette shape does not.

2. Cell histogramming (orientation binning). The window is partitioned into non-overlapping cells of `cell_size` pixels (`self._cell_size`, default $(2,2)$ — deliberately tiny because a person silhouette at native MLX90640/Waveshare thermal resolution spans only a handful of pixels, so a single pixel is already a meaningful spatial unit). Within each cell, a histogram of `orientations = 9` bins is built over $[0^\circ,180^\circ)$ by **soft-binning**: each pixel's gradient magnitude $m(x,y)$ is distributed (via linear interpolation) between its two nearest orientation bins in proportion to angular proximity, so a pixel does not have to fall exactly on a bin center to contribute:

$$h_{\text{cell}}(b) = \sum_{(x,y)\,\in\,\text{cell}} m(x,y)\cdot w_b(\theta(x,y)), \qquad b=1,\dots,9,$$

where w_b is the (triangular) interpolation weight for bin b. This produces one 9-D vector per cell.

3. Block normalization. Cells are grouped into overlapping blocks of `block_size_cells` (default $(2,2)$ cells per block, `cells_per_block=(2,2)`), and the concatenated cell histograms within each block are normalized with **L2-Hys** — L2 normalize, clip to a maximum component value (0.2, skimage's default), then re-normalize:

$$v \leftarrow \frac{v}{\|v\|_2+\epsilon}, \qquad v \leftarrow \min(v, 0.2), \qquad v \leftarrow \frac{v}{\|v\|_2+\epsilon}.$$

Block-level (rather than whole-window) normalization makes the descriptor **locally** robust to contrast/illumination gradients across the window — one very hot sub-region does not wash out the gradient structure of a cooler sub-region elsewhere in the same window — which is exactly why the code additionally *pre*-normalizes each window to zero mean/unit variance (`_normalize_patch`) before computing HOG, so the detector is insensitive to the absolute ambient temperature of the scene

as well as to local contrast.

The concatenation of all block-normalized histograms is the final HOG feature vector $X \in \mathbb{R}^{d}$, with d determined by `(window_size / cell_size)` and `block_size_cells` (`self._n_features`, recorded at fit time and validated at load time).

3.3 The linear SVM: primal objective and decision rule

The trained classifier is an `sklearn.svm.LinearSVC`, which — unlike SVC (§2) — solves the **primal** directly via the `liblinear` coordinate-descent solver rather than forming a Lagrangian dual. `LinearSVC` minimizes the `L_2`-regularized (squared) hinge-loss objective

$$\min_{w,b}\ \frac{1}{2}\|w\|^2 \;+\; C\sum_{i=1}^n \max\bigl(0,\ 1-y_i(w^\top X_i+b)\bigr) \tag{11}$$

directly over $(w,b) \in \mathbb{R}^{d+1}$ — the same *loss-plus-penalty* form used to motivate the dual in §2.2, but here it is the objective actually being minimized by the solver, not an equivalent reformulation of a QP that is then dualized. Because there is no kernel and no need to express the model as a weighted sum over training points, `liblinear` never forms Lagrange multipliers or a Gram matrix; it works directly in the d-dimensional primal weight space, which scales far better than kernel SVC when d is in the hundreds (as HOG feature vectors are) and n (training windows) is large. **This is why the KKT/dual machinery of §2.3–2.5 is not the operative framework for `HOGSVMDetector`**: KKT conditions and strong duality remain true statements about problem (11) (it is still a convex QP-equivalent program with affine constraints once slacks are reintroduced), but the solver never constructs or exploits them — `liblinear` uses primal coordinate descent / trust-region Newton steps directly on (w,b), so there are no α_i, no support vectors, and no kernel to discuss for this detector.

The fitted decision function is the plain affine score

$$f(X) = w^\top X + b, \qquad \hat y = \operatorname{sign} f(X),$$

which the code evaluates once per window as `self._svm.decision_function(features.reshape(1,-1))`, thresholds against `score_threshold`, and finally maps through a sigmoid $\sigma(f(X))$ purely to populate `Detection.score` with a quasi-probability in $(0,1)$ — the sigmoid here is a monotonic rescaling for reporting, not a calibrated probability model (contrast with Platt scaling in §2.7, which is fit to data).

3.4 Why a linear kernel suffices/is preferred here

Three factors favor the linear model over the RBF approach used for fire (§2):

1. **Dimensionality.** HOG descriptors are high-dimensional (the window is deliberately sized from a physical person silhouette via `SensorProfile.physical_pixel_size_m`, then snapped to a cell-size multiple — typically producing d in the hundreds even at thermal resolution). In high dimensions, a rich-enough set of *linear* projections of the input (here, the 9-orientation-bin histogram) already encodes most of the discriminative shape information that a nonlinear kernel would otherwise have to reconstruct implicitly — the classic empirical finding (Dalal & Triggs, 2005) that motivated

pairing HOG specifically with a linear SVM. 2. **No kernel trick needed.** Cover's theorem intuition: as feature dimensionality grows, a linear separator becomes increasingly likely to exist (or nearly exist) for a fixed number of training points, since the VC dimension of linear separators grows with d while the training-set size is fixed by how many labeled person/background windows are available. 3. **Speed at inference.** The sliding-window multi-scale search (§3.6) evaluates the decision function at every window position and every pyramid scale — potentially thousands of evaluations per frame. A linear score `w^\top X+b` is a single dot product per window; an RBF score would require a kernel evaluation against every stored support vector, which is asymptotically far more expensive precisely where this detector needs to be cheap.

3.5 Implementation notes: window sizing and normalization

- `_resolve_window_size` derives the pixel window from a *physical* target size (`physical_person_size_m`, default `(0.9\,\text{m}, 0.45\,\text{m})`) and an assumed viewing distance via `SensorProfile.physical_pixel_size_m(distance_m)`, then `_snap_to_multiple` rounds down to an integer multiple of `cell_size` (required so HOG receives an integer cell grid) with a floor of `2\times` cell size per axis (so a `2\times2`-cell block fits). This is why `resolution_behavior = "parameterized"`: the window/cell geometry auto-scales per sensor profile, but a checkpoint's learned `(w,b)` does not transfer between profiles because the feature dimensionality differs.
- `_normalize_patch` zero-means and unit-variances every window (both training patches and inference windows) before HOG is computed — this is what makes the detector insensitive to the *ambient* temperature of the room without needing to know it, complementing HOG's own block-level contrast normalization (§3.2).

3.6 Sliding-window multi-scale pyramid

`predict()` iterates over `pyramid_scales` (default `(1.0,)`, but configurable to e.g. `\{0.75,1.0,1.25,1.5\}\times`, as evaluated in `reports/human_detection_report.tex`), resizing the frame at each scale (`cv2.resize`, `INTER_LINEAR` when upscaling / `INTER_AREA` when downscaling) and sliding the **fixed-size** detection window over the resized frame with `stride`. A window at scale `s`, positioned at `(x_0,y_0)` in the resized frame, maps back to the original frame's coordinates by dividing by `s`: `(x0/scale, y0/scale, ww/scale, wh/scale)`. Running the same fixed window over multiple resized copies of the image is algorithmically equivalent to running variable-size windows over one fixed image, but is simpler to implement given a HOG extractor tied to one cell/block geometry; it lets the detector match people who appear larger or smaller than the training window's implicit physical scale (e.g., closer/farther from the sensor) — at the empirically documented cost of somewhat more false positives per `reports/human_detection_report.tex`'s pyramid ablation, since more windows are scored in total. All boxes across all scales are pooled and a single greedy NMS pass (`_greedy_nms`, IoU threshold `nms_iou_threshold`) removes duplicate detections of the same person.

---

4. MobileNetSSDDetector — human_detection/mobilenet_ssd.py, mobilenet_ssd_model.py, mobilenet_ssd_anchors.py

4.1 Formulation

MobileNetSSDDetector is a single-stage, anchor-based deep detector (SSD — Liu et al., 2016 — style) built on a small MobileNet backbone (MicroMobileNet), run directly on the native sensor resolution (no upsampling to 300×300). Two feature maps F1 (fine) and F2 (coarse) are tapped from the backbone; each spatial cell of each map predicts, for a fixed set of K anchor boxes anchored at that cell, 4 box-regression offsets and num_classes = 2 (background/person) class logits.

4.2 Depthwise-separable convolutions (backbone)

Each DepthwiseSeparableBlock factors a standard convolution into two cheaper steps, exactly as implemented:

$$\text{depthwise: } y_c(x,y) = \sum_{(dx,dy)\in\mathcal{K}} w_c(dx,dy)\, x_c(x+dx,y+dy) \quad \text{(one filter per input channel } c),$$

$$\text{pointwise: } z_{c'}(x,y) = \sum_c u_{c',c}\, y_c(x,y) \quad \text{(1×1 conv, channel mixing only).}$$

A standard 3×3 convolution from C_{in} to C_{out} channels costs $3\cdot3\cdot C_{\text{in}}\cdot C_{\text{out}}$ multiply-accumulates per output pixel; the depthwise+pointwise factorization costs $3\cdot3\cdot C_{\text{in}} + C_{\text{in}}\cdot C_{\text{out}}$, a ratio of $\tfrac{1}{C_{\text{out}}}+\tfrac{1}{9}$ — roughly an order of magnitude cheaper for the channel counts used here (adapted from reports/checkpoint_report.tex, "Why depthwise-separable convolutions"). This is what keeps MicroMobileNet cheap enough to run per-frame at the target frame rate.

4.3 Anchor generation

generate_anchors places, at every cell (i,j) of a feature map of size $f_h\times f_w$ over an input of size $i_h\times i_w$, $K=|\text{aspect\_ratios}|$ anchors centered at the cell's projected input-pixel location:

$$(cx,cy) = \Bigl((j+\tfrac12)\cdot\tfrac{i_w}{f_w},\ (i+\tfrac12)\cdot\tfrac{i_h}{f_h}\Bigr), \qquad w = \texttt{anchor\_width}, \qquad h = \frac{w}{\texttt{aspect\_ratio}},$$

for each aspect ratio in aspect_ratios = (1.0, 0.5, 0.33) (square, 2×-tall, 3×-tall — matching a standing person's silhouette). F1 and F2 use different anchor_widths per profile (e.g. Waveshare: 10 px at F1, 16 px at F2), so the two feature maps jointly cover a range of apparent person sizes without an image pyramid: F1's finer grid and smaller anchors catch distant/small people, F2's coarser grid and larger anchors catch close/large ones (MicroMobileNetSSD.build_anchors concatenates both levels' anchors into one $(N_{\text{anchors}},4)$ buffer in (cx,cy,w,h)

form).

4.4 IoU-based anchor matching (positive/negative assignment)

For a ground-truth box set \{g_k\} and anchor set \{a_m\}, pairwise_iou computes, in corner form (x_1,y_1,x_2,y_2) derived from (cx,cy,w,h) via cxcywh_to_xyxy,

$$\mathrm{IoU}(a,g) = \frac{|\,a\cap g\,|}{|\,a\cup g\,|} = \frac{\max(0,\min(x_2^a,x_2^g)-\max(x_1^a,x_1^g))\cdot\max(0,\min(y_2^a,y_2^g)-\max(y_1^a,y_1^g))}{|a|+|g|-|\,a\cap g\,|}.$$

match_anchors_to_targets then assigns, for each anchor m, a label via the standard SSD rule implemented exactly as follows:

1. Every anchor's best-matching ground truth is g^*(m) = \arg\max_k \mathrm{IoU}(a_m,g_k), with score \mathrm{IoU}^*(m). 2. Anchor m is **positive**, matched to g^*(m), if \mathrm{IoU}^*(m) \ge iou_pos_threshold (0.5). 3. Anchor m is **negative** (background) if \mathrm{IoU}^*(m) < iou_neg_threshold (0.4); anchors in [0.4,0.5) are **ignored** (contribute to neither loss term). 4. **Forced positive override**: for every ground-truth box g_k, the single anchor \arg\max_m \mathrm{IoU}(a_m,g_k) is *always* forced positive regardless of its raw IoU — this guarantees every labeled person has at least one responsible anchor, even a small/oddly-shaped person whose best IoU with any anchor never reaches 0.5.

4.5 The multi-task SSD loss

Given the matching of §4.4, ssd_loss computes two terms per training sample, matching the code exactly.

Localization loss (positives only). For each positive anchor m matched to g^*(m), the target is the **encoded offset** of the ground-truth box relative to the anchor (not the absolute box) — this is what encode_boxes computes:

$$\Delta_{cx} = \frac{cx_g - cx_a}{w_a \cdot v_{xy}},\quad \Delta_{cy} = \frac{cy_g - cy_a}{h_a \cdot v_{xy}},\quad \Delta_{\log w} = \frac{\log(w_g/w_a)}{v_{wh}},\quad \Delta_{\log h} = \frac{\log(h_g/h_a)}{v_{wh}},$$

with the standard SSD variance scales v_{xy}=0.1, v_{wh}=0.2 (BBOX_VARIANCE_XY, BBOX_VARIANCE_WH) rescaling the raw offsets so all four regression targets have comparable magnitude/gradient scale during training — the network's box head predicts these four numbers directly, and decode_boxes is the exact algebraic inverse used at inference (§4.7). The loss on these offsets is **Smooth-L1** (Huber),

$$\mathrm{SmoothL1}(e) = \begin{cases} \tfrac12 e^2, & |e|<1 \\ |e|-\tfrac12, & |e|\ge 1,\end{cases} \qquad \mathcal{L}_{\text{loc}} = \sum_{m\,\text{positive}} \mathrm{SmoothL1}\bigl(\text{pred}_m - \Delta(g^*(m),a_m)\bigr),$$

implemented as `F.smooth_l1_loss(..., reduction="sum")`. Smooth-L1 is quadratic (like squared error) for small residuals — precise fine-tuning near the target — but linear for large residuals, so a handful of badly-matched anchors early in training cannot produce the exploding gradients that pure squared error would (adapted from `reports/checkpoint_report.tex`).

Classification loss (positives + hard negatives). Every anchor's binary target is `c_m = 1` if matched positive, `0` otherwise (background):

$$\mathcal{L}_{\text{cls}} = \sum_{m\,\in\,\text{keep}} \mathrm{CE}(\text{cls\_preds}_m,\ c_m), \qquad \mathrm{CE}(\ell,c) = -\log\operatorname{softmax}(\ell)_c,$$

implemented as `F.cross_entropy(..., reduction="sum")` over the anchors selected by hard-negative mining (§4.6).

Total loss, normalized by the number of positive anchors in the batch (so the loss scale is independent of how many people are in a given frame):

$$\mathcal{L} = \frac{\mathcal{L}_{\text{cls}}}{\max(N_{\text{pos}},1)} + \lambda\cdot\frac{\mathcal{L}_{\text{loc}}}{\max(N_{\text{pos}},1)}, \qquad \lambda = \texttt{loc\_loss\_weight} = 1.0,$$

exactly the `cls_loss + loc_loss_weight * loc_loss` computed in `ssd_loss`. This is a genuinely **multi-task** loss because it trains the two conv heads (`box_head`, `cls_head`) jointly, sharing the backbone — the classification term teaches *which* anchors contain a person, the localization term teaches *where* the box should sit relative to the matched anchor.

4.6 Hard-negative mining

With `num_classes=2` and typically hundreds to over a thousand anchors (`720` for MLX90640, `1110` for Waveshare, per the module docstring) but usually only a handful of people per frame, the vast majority of anchors are background. Training on *all* negatives would let the trivial "always predict background" solution dominate the loss. `hard_negative_mining` instead:

1. Computes each anchor's log-softmax background score `\log p(\text{class}=0\mid \text{anchor})`. 2. Restricts attention to anchors labeled negative (`matches == NEG_GT`). 3. Keeps only the `n_neg_keep = min(neg_pos_ratio * n_pos, n_negatives_available)` **hardest** negatives, i.e. those with the *lowest* background log-probability — the negatives the network is currently most confidently (and wrongly) calling "person-like." 4. Returns `pos_mask | keep_neg` as the boolean mask fed into `\mathcal{L}_{\text{cls}}`'s sum.

With `neg_pos_ratio = 3.0` (the constructor default), this fixes the classification loss's positive:negative ratio at `1{:}3` regardless of how imbalanced the raw anchor population is, concentrating gradient signal on the negatives that are actually informative (near-misses) rather than the overwhelming majority of easy true negatives far from any person.

4.7 Inference: decoding and NMS

At inference, `decode_predictions` inverts the encoding of §4.5 (`decode_boxes`, the algebraic inverse of `encode_boxes`) to recover absolute `(cx,cy,w,h)` boxes from the network's raw offset predictions, converts to corner form, applies `softmax` to the class logits to get calibrated-in-training-objective probabilities, and keeps boxes with `P(\text{person}) > score_threshold` (`0.5` default). Because neighboring anchors and both feature-map levels typically all fire on the same physical person, `nms_xyxy` — the same greedy algorithm as `_greedy_nms` in §3.6 — then collapses duplicate boxes: sort by score, keep the top box, discard every remaining box with `\mathrm{IoU} > nms_iou_threshold` (`0.3`) against it, and repeat.

4.8 Implementation notes

- `resolution_behavior = "fixed"`: because the backbone's channel/stride schedule and the anchor grid's absolute pixel geometry are baked into `BackboneConfig`/`SSDConfig` per profile (`MLX90640_BACKBONE`/`_SSD` vs. `WAVESHARE_BACKBONE`/`_SSD`), a checkpoint trained for one sensor's `(H,W)` cannot be loaded for the other — two independent checkpoints are required, unlike the resolution-invariant `FireSVMDetector` (§2) or the parameterized `HOGSVMDetector` (§3).
- Training uses per-frame z-score normalization (`_augment`/`_normalize`: `(x-\mu)/(\sigma+10^{-6})`) plus random horizontal flip and small translation augmentation (with box coordinates transformed accordingly) — see `MobileNetSSDDetector._augment`.
- `Adam` optimizer with gradient-norm clipping (`clip_grad_norm_(max_norm=5.0)`) is used for the (non-convex) network training, in sharp contrast to the convex QP of §2 — this is a fundamentally different optimization regime (stochastic gradient descent on a non-convex loss surface, no KKT/global-optimality guarantee) than the SVM sections.

---

5. GeometricContactDetector — `contact_detection/geometric.py`, `multi_view/homography.py`, `multi_view/fusion.py`

5.1 Formulation

`GeometricContactDetector` is a deterministic, non-learned pipeline: it projects each camera's 2-D bounding-box detections onto a shared floor plane via a calibrated homography, fuses cross-camera projections into unique "actor" nodes, and flags contact when two actors' floor-plane positions fall within a fixed distance `\delta`.

5.2 Homography via DLT + SVD

Setup. A planar homography `H_k \in \mathbb{R}^{3\times3}` maps a camera-k image point `(u,v)` (homogeneous `[u,v,1]^\top`) to a floor-plane world point `(X_w,Y_w)` (homogeneous `[X_w,Y_w,1]^\top`, up to scale) whenever the imaged points lie on a single plane (here, `Z=0`, the floor):

$$w\begin{bmatrix}X_w\\Y_w\\1\end{bmatrix} \cong H_k\begin{bmatrix}u\\v\\1\end{bmatrix}. \tag{12}$$

Deriving the DLT linear system. Write H_k row-wise as h_1^\top, h_2^\top, h_3^\top (rows of H_k) so that X_w = (h_1^\top p)/(h_3^\top p), Y_w = (h_2^\top p)/(h_3^\top p) with p=[u,v,1]^\top. Clearing denominators,

$$h_1^\top p - X_w\, h_3^\top p = 0, \qquad h_2^\top p - Y_w\, h_3^\top p = 0,$$

which, written out in the 9 unknowns h = \mathrm{vec}(H_k), are exactly the two rows the code assembles per correspondence:

$$\text{row}_1 = [-u,-v,-1,\ 0,0,0,\ u X_w,\ v X_w,\ X_w]\cdot h = 0, \qquad \text{row}_2 = [0,0,0,\ -u,-v,-1,\ u Y_w,\ v Y_w,\ Y_w]\cdot h = 0. \tag{13}$$

Each of the N \ge 4 calibration correspondences (u_i,v_i)\leftrightarrow(X_{w,i},Y_{w,i}) (heated targets at known floor positions — "Hot-Point Calibration," since checkerboard patterns have no thermal contrast) contributes both rows, stacking into a 2N\times9 homogeneous system Ah=0.

Why SVD gives the solution. Ah=0 has the trivial solution h=0, which is useless; the meaningful constrained problem is \min_h \|Ah\|^2 subject to \|h\|=1 (fixing the scale ambiguity inherent to homogeneous coordinates — H_k and \lambda H_k represent the same projective map). Writing A = U\Sigma V^\top (the SVD), \|Ah\|^2 = \|U\Sigma V^\top h\|^2 = \|\Sigma V^\top h\|^2 (since U is orthogonal), which is minimized over unit-norm h by setting h equal to the right-singular vector corresponding to \Sigma's **smallest** singular value — because \Sigma V^\top h then has all its energy concentrated in the smallest diagonal entry of \Sigma. This is exactly _dlt_solve's Vt[-1] (the last row of V^\top, i.e. the smallest-singular-value right singular vector of np.linalg.svd(A)), reshaped to 3\times3 and rescaled so H[2,2]=1 (an arbitrary but standard normalization removing the remaining scale ambiguity). With more than the minimal 4 correspondences, this is simultaneously the **least-squares** solution to the over-determined homogeneous system — the SVD approach degrades gracefully from an exact solve to a best-fit as N grows, which is why the docstring recommends more than the minimum 4 points for accuracy.

5.3 Foot-point projection

Given a detection's bounding box (x,y,w,h) (top-left corner convention), the code approximates its floor contact point by the **bottom-center** of the box,

$$P_{\text{foot}} = \Bigl(x+\tfrac{w}{2},\ y+h\Bigr)$$

(Detection.foot_point) — physically justified because a standing or seated person's bounding box, in a top/side thermal view, has its base at the floor and its horizontal center roughly aligned with the body's vertical axis, so the bottom-center is the best single-point proxy for "where the person is touching the ground" available from a 2-D box alone. project_foot_point then applies Eq. (12) directly: p_{\text{world}} = H_k\,[u,v,1]^\top, followed by the homogeneous divide X_w = p_{\text{world}}[0]/p_{\text{world}}[2], Y_w = p_{\text{world}}[1]/p_{\text{world}}[2] (guarded against |w|<10^{-9}, i.e. points that map to infinity).

5.4 Multi-view fusion: clustering and validation

`fuse_detections` (§`multi_view/fusion.py`) projects every camera's detections to world points, then:

1. **Greedy single-linkage clustering** (`_greedy_cluster`): a world point joins the first existing cluster containing any member within `epsilon_m` (`\varepsilon`, default `0.5` m) of it; otherwise it starts a new cluster. This groups projections of the *same* physical person seen from different cameras. 2. **Validation** (`_validate_cluster`), applying the physical prior that a genuine person should be corroborated by more than one camera:

- `N=1` (single-camera): discarded — no cross-camera corroboration, so a lone projection cannot be distinguished from a false detection or a projection error.
- `N=2`: accepted iff the pairwise distance is `<\varepsilon` (consistency check).
- `N=3`: **outlier removal** — if exactly one point is `>\varepsilon` from *both* others while the remaining pair is mutually `<\varepsilon`, that point is dropped and the consistent pair is kept; if all three are mutually consistent, all three are kept; otherwise (no consistent pair) the cluster is discarded entirely.
- `N>3`: all points kept (already enforced to be within `\varepsilon` pairwise-transitively by the clustering step).

3. Each validated cluster's centroid `(\bar X,\bar Y) = \bigl(\tfrac1n\sum x_i,\ \tfrac1n\sum y_i\bigr)` becomes one `ActorPosition`, with `confidence` the mean of its members' detector confidences and `source_camera_ids` recording which cameras contributed.

5.5 The Euclidean-distance contact rule

Once unique actors' floor positions are known, `GeometricContactDetector.predict` computes, for every actor pair `(i,j)`,

$$D_{ij} = \sqrt{(X_i-X_j)^2 + (Y_i-Y_j)^2}, \qquad \text{contact}(i,j) \iff D_{ij} < \delta_m$$

(`delta_m`, default `0.5` m) — a direct thresholded Euclidean distance on the projected floor plane.

5.6 Why a simple threshold is a valid decision rule here

Physical contact between two people is, almost by definition, a statement about the distance between their bodies falling below the scale of a human body — two people cannot be "touching" while their centroids (or, here, foot-points) are meters apart. Once the earlier pipeline stages (detection → foot-point extraction → homographic projection → cross-camera fusion) have already converted image-plane pixel coordinates into a single, metric, common floor-plane coordinate system, "distance below a body-scale threshold" is not an approximation of the physical contact criterion — it *is* the physical contact criterion, restated in the coordinate system the pipeline has arranged to make it directly checkable. This is why no learned classifier is needed at this final stage (`is_trainable = False`; `fit()` is a no-op): the earlier geometric/homographic stages have already done the work of turning "contact" into a metric-distance question, and a threshold in metric units is the natural (and, given a correctly calibrated homography, essentially exact) answer to that question. The two free parameters — `\varepsilon` (clustering/consistency) and `\delta` (contact) — are tuned

independently because they answer different questions: \varepsilon bounds the expected *projection noise* of the homography (how far apart two projections of the *same* person's foot-point can legitimately land due to calibration error), while \delta bounds the physical extent of *two different* people's bodies when in contact; conflating them would either over-merge distinct nearby people (if \varepsilon were set to the larger \delta-scale value) or under-merge noisy projections of one person (if \delta were set to the smaller \varepsilon-scale value).

---

6. MVSTGCNDetector — contact_detection/mv_stgcn.py

6.1 Formulation

MVSTGCNDetector extends the geometric front-end of §5 with a learned spatiotemporal classifier: a sliding window of T=16 timesteps of actor-graphs is fed to a small Graph Convolutional Network (_ThermalSTGCN) that outputs a 2-class (contact/no-contact) probability, gated by a persistence rule before alerting.

6.2 Node features (7-D)

For each of up to max_actors (N=5 default) actors at a given timestep, _actors_to_node_features builds the 7-D vector

$$f = \bigl[\,X_{\text{norm}},\ Y_{\text{norm}},\ v_x,\ v_y,\ T_{\max,\text{norm}},\ T_{\text{mean,norm}},\ \text{area}\,\bigr] \in \mathbb{R}^7,$$

with world position normalized by an assumed room scale (X/5, Y/5, "room ≈ 5 m"), velocity from the per-camera Kalman tracker's constant-velocity state (scaled by 1/10), and thermal statistics scaled by 1/100. Padded (absent) actor slots get an all-zero feature row and mask=0; the mask is threaded through every subsequent computation so padding never influences the graph convolution or the final pooled representation.

6.3 Adaptive (Gaussian-distance) graph adjacency

For a batch of N nodes at one timestep with world positions \{p_n\}, _AdaptiveGCNLayer.forward computes the squared pairwise distance matrix

$$d^2_{ij} = \|p_i-p_j\|^2$$

(computed once per timestep in _ThermalSTGCN.forward via broadcasted subtraction over the flattened (B\!\cdot\!T, N, 2) position tensor) and derives a **row-stochastic, distance-decaying adjacency**:

$$A_{ij} = \frac{\exp(-d^2_{ij}/\sigma^2)}{\sum_{j'} \exp(-d^2_{ij'}/\sigma^2)} = \operatorname{softmax}_j\Bigl(-\frac{d^2_{ij}}{\sigma^2}\Bigr), \tag{14}$$

exactly `F.softmax(-dist2/sigma2, dim=2)` in the code, with padded columns masked to `-\infty` before the softmax so they receive zero weight. This is a **Gaussian-kernel graph**: nearby actors (small `d_{ij}`) get large edge weight, distant actors get exponentially suppressed weight — the natural continuous relaxation of "connect actors that are near each other," parameterized by the learnable/fixed bandwidth `\sigma` (`sigma=1.0` default in the layer). Unlike a fixed geometric graph (e.g. connect-if-d<`\delta`, as in §5.5), this adjacency is **soft and differentiable**, allowing gradient-based training to shape how strongly proximity translates into information flow.

6.4 Propagation rule

Self-loops are added (`\hat A = A + I`) so every node retains its own features in the aggregation, then **symmetric-style row normalization** is applied via the node degree `d_i = \sum_j \hat A_{ij}`:

$$\bar A_{ij} = \frac{\hat A_{ij}}{d_i}, \qquad \text{(the code's } A_{\text{norm}} = \hat A / \deg\text{, i.e. } D^{-1}\hat A \text{, a row-normalized rather than symmetric } D^{-1/2}\hat A D^{-1/2}\text{ variant),}$$

and one GCN layer computes

$$H^{(\ell+1)} = \operatorname{ReLU}\!\bigl(\bar A\, H^{(\ell)}\, W^{(\ell)}\bigr), \qquad H^{(0)} = f \ \text{(the 7-D node features)}, \tag{15}$$

which is the standard Kipf–Welling graph-convolution propagation rule (`support = A_norm @ x` then `W(support)` then `ReLU`, masked by the actor-presence mask so padded nodes stay zero), stacked `n_gcn_layers` times (default 2). Equation (15) is the exact graph analogue of a normal feed-forward layer `H^{(\ell+1)}=\operatorname{ReLU}(H^{(\ell)}W^{(\ell)})`, with the crucial difference that each node's pre-activation is first **averaged with its (adjacency-weighted) neighbors** before the linear map — this is what lets a node's updated representation depend on nearby actors' state, which is the entire point of using a graph rather than treating each actor independently.

6.5 Spatiotemporal structure over the T=16 window

The full architecture is genuinely **spatiotemporal**, but not via explicit temporal graph edges: at each of the `T=16` timesteps in the sliding window, the `N` actors form a **fully-connected spatial graph** (every pair gets an adjacency weight from Eq. 14; there is no discrete "connect/don't connect" spatial edge decision — the Gaussian kernel makes every pair weakly-to-strongly connected depending on distance). The **temporal** dimension is handled not by an explicit temporal adjacency but by **weight-sharing across time combined with global pooling**: `_ThermalSTGCN.forward` reshapes `(B,T,N,\cdot) \to (B\!\cdot\!T,N,\cdot)`, applies the *same* GCN layers independently to every timestep's spatial graph (this is effectively `T` chained/replicated spatial-only graphs, not a spatiotemporal graph with per-node temporal self-edges — the "temporal chain" connecting the same actor across consecutive timesteps is realized only implicitly, through the Kalman-filtered velocity feature `v_x,v_y` carried in each node's own feature vector, and explicitly through the final pooling step), then reshapes back to `(B,T,N,\text{hidden})` and performs a **masked global average

pool over both T and N**:

$$z = \frac{\sum_{t,n} \text{mask}_{t,n}\cdot H^{(\text{final})}_{t,n}}{\sum_{t,n}\text{mask}_{t,n}} \in \mathbb{R}^{\text{hidden\_dim}},$$

which the classifier head ($\text{Linear}(\text{hidden\_dim},32) \to \text{ReLU} \to \text{Linear}(32,2)$) maps to 2 class logits, softmax-normalized to a contact probability. The sliding window of T=16 gives the GCN's temporal context — a single instant's actor graph cannot capture *approach and interaction over time*, but 16 consecutive fully-connected spatial graphs, each carrying velocity, jointly encode a short spatiotemporal trajectory even though the graph convolution itself only mixes information within a timestep.

6.6 Implementation notes

- `resolution_behavior = "invariant"`: all graph computation operates on world-space (metric) coordinates from the homographic front-end (§5), so — like `GeometricContactDetector` — the same trained model works across sensor profiles.
- The persistence gate (`conf_threshold`, `persistence_frames`) requires the softmax contact probability to exceed threshold for `persistence_frames` *consecutive* predict() calls before actually alerting, suppressing single-frame noise in the GCN's output — a rule applied on top of, not inside, the network.
- Training minimizes `nn.CrossEntropyLoss` (optionally class-weighted for the contact-minority imbalance, per `data/DATASET_NOTES.md`) via `Adam`, i.e. ordinary non-convex SGD-family training — the KKT/duality framework of §2 does not apply to this model for the same reason it does not apply to §4's SSD network: the objective is a non-convex function of the network weights.

---

7. ThermoX3DDetector — `contact_detection/thermo_x3d.py`

7.1 Formulation

`ThermoX3DDetector` is the pixel-based, geometry-free failsafe contact detector: it processes raw `(T,H,W)` thermal volumes from each camera directly through a small factorized 3-D CNN, with **no bounding boxes, no homography, and no per-camera fusion** — a design response to the documented failure mode of §5–6 where close contact causes two people's bounding boxes to visually merge, leaving the box-based detectors with a single blob and no way to represent "two actors in contact" (`mv_stgcn.py`/`geometric.py` both depend on the upstream `HumanDetector` correctly separating actors; Thermo-X3D does not).

7.2 3-D convolution as a generalization of 2-D convolution

A standard 2-D convolution on an image $I(y,x)$ with kernel w of spatial extent (k_h,k_w) produces

$$O(y,x) = \sum_{dy=-k_h/2}^{k_h/2}\sum_{dx=-k_w/2}^{k_w/2} w(dy,dx)\, I(y+dy,x+dx).$$

A **3-D convolution** simply extends the kernel and the sliding window along a third axis — here, time `t` — over a volume `V(t,y,x)` with a kernel of extent `(k_t,k_h,k_w)`:

$$O(t,y,x) = \sum_{dt=-k_t/2}^{k_t/2}\sum_{dy}\sum_{dx} w(dt,dy,dx)\, V(t+dt,\,y+dy,\,x+dx). \tag{16}$$

Equation (16) reduces exactly to the 2-D case when `k_t=1` (as the `_MicroX3DStream.stem`'s `\texttt{kernel\_size=(1,3,3)}` does) — a 3-D convolution is a strict generalization of a 2-D convolution, obtained by allowing the receptive field to span multiple *frames*, not just multiple *pixels within one frame*; this is what lets the network respond to genuinely spatiotemporal patterns (two hot blobs' shapes merging over several frames) rather than only to any single frame's static appearance.

7.3 (2+1)D factorization

Rather than a full 3-D kernel `w(dt,dy,dx)` with `k_t\cdot k_h\cdot k_w` free parameters per input/output channel pair, `_FactorizedResBlock` **factorizes** each block into a purely spatial 3-D convolution (temporal extent 1) followed by a purely temporal 3-D convolution (spatial extent 1), matching the code precisely:

$$\text{spatial: } \texttt{Conv3d}(k_t{=}1,\ k_h{=}3,\ k_w{=}3,\ \text{stride}=(1,s,s)) \;\to\; \text{BN} \;\to\; \text{ReLU},$$

$$\text{temporal: } \texttt{Conv3d}(k_t{=}3,\ k_h{=}1,\ k_w{=}1,\ \text{stride}=1) \;\to\; \text{BN} \;\to\; \text{ReLU},$$

i.e. $w(dt,dy,dx) \approx w_{\text{sp}}(dy,dx)\cdot w_{\text{temp}}(dt)$ — a rank-1-like separable approximation of the full 3-D kernel. This factorization: (i) reduces parameters/FLOPs from `O(k_t k_h k_w)` to `O(k_h k_w + k_t)` per channel pair, (ii) inserts an extra BN+ReLU nonlinearity between the spatial and temporal steps (more expressive than one linear 3-D conv for the same or lower parameter count), and (iii) — since every individual op is now either a plain 2-D-shaped convolution (`1\times3\times3`) or a plain 1-D-shaped convolution (`3\times1\times1`) — maps well onto CPU inference kernels that are heavily optimized for 2-D convolution, which matters for the edge (Raspberry Pi CPU) deployment target (adapted from `reports/checkpoint_report.tex`, "(2+1)D-factorized"). Note the current implementation's window is `T=16` (`ThermoX3DDetector.__init__(T=16)`) with per-corpus global normalization (§7.5), differing from the `T=5`/rolling-p25-background configuration documented for the deployed "T5v2" checkpoint variant in `reports/checkpoint_report.tex` — the derivation above is architecture-level and applies to either `T`.

Each residual block also applies a **skip connection** — identity if channel count and stride are unchanged, else a `1\times1\times1` projection conv + BN — added after the (attention-gated) spatial+temporal path and before a final ReLU: $\text{out} = \operatorname{ReLU}(\text{skip}(x) + \text{attention}(\text{temporal}(\text{spatial}(x))))$, the standard ResNet-style residual formulation lifted to 3-D, which keeps gradients well-behaved through the three stacked blocks (24→48→96 channels, spatial stride 2 at blocks 2 and 3).

7.4 CBAM-style thermal attention

Before the residual sum, `_ThermalAttention` reweights the block's output feature map using channel and spatial gates, matching the code:

$$\text{Channel: } M_c = \sigma\bigl(\mathrm{MLP}(\mathrm{AvgPool}_{T,H,W}(X)) + \mathrm{MLP}(\mathrm{MaxPool}_{T,H,W}(X))\bigr) \in (0,1)^{C}, \qquad X' = X\cdot M_c,$$

$$\text{Spatial: } M_s = \sigma\bigl(\mathrm{Conv}_{1\times7\times7}\bigl([\,\mathrm{AvgPool}_C(X');\,\mathrm{MaxPool}_C(X')\,]\bigr)\bigr), \qquad X'' = X'\cdot M_s,$$

with the same 2-layer-MLP-with-reduction-4 channel gate and the same 2-channel-input `7\times7` spatial gate as the CBAM formulation (Woo et al., 2018), adapted to 3-D pooling (over `(T,H,W)` for the channel gate, over the channel axis for the spatial gate). Using *both* average- and max-pooling in the channel gate lets the network respond both to a region's overall thermal energy (average) and to its single hottest voxel (max) — appropriate for a signal (contact) where both "two warm blobs merging" (average-detectable) and "a sharp new hot edge appearing where two bodies meet" (max-detectable) are informative. This attention is `O(C)+O(HW)` per block (a shared small MLP over pooled statistics, plus one `7\times7` conv over an already-pooled 2-channel map) — categorically cheaper than quadratic-cost self-attention, since it never forms an `N\times N` token-similarity matrix (adapted from `reports/checkpoint_report.tex`, "Why attention here is cheap").

7.5 Global normalization

Unlike the per-frame normalization used by `MobileNetSSDDetector` (§4.8), `ThermoX3DDetector.fit()` computes **corpus-wide** normalization statistics once, over every pixel of every frame in the training set:

$$\mu_{\text{global}} = \frac{1}{M}\sum_{k=1}^{M} v_k, \qquad \sigma_{\text{global}} = \sqrt{\frac{1}{M}\sum_k (v_k-\mu_{\text{global}})^2} + 10^{-6},$$

(`self._global_mean`, `self._global_std`, over the concatenation of all `frame.data.ravel()` across the training set) and applies `(x-\mu_{\text{global}})/\sigma_{\text{global}}` identically at training and inference (`_normalise`, persisted in `_state_dict`/`_load_state_dict` so inference exactly reproduces the training distribution). This is the correct choice specifically because the network's job is to detect a spatiotemporal *pattern* (bodies merging/interacting) rather than to be invariant to each individual clip's own brightness — per-frame normalization would erase genuine cross-frame temperature differences (e.g., a person's silhouette warming or two bodies' thermal signatures blending as they make contact) that are exactly the signal this detector is trying to learn; a single fixed global affine map preserves those relative differences throughout a T-frame volume.

7.6 Late fusion and the softmax classification head

Each camera stream is encoded **independently** by its own `_MicroX3DStream` (three separate instances in `nn.ModuleList`, no shared weights) down to a 128-D vector via

`AdaptiveAvgPool3d(1)` (**global average pooling** — collapsing the entire `(T,H,W)` volume to one value per channel, appropriate because contact is a spatially/temporally *distributed* event rather than one localized peak, unlike a use-case where global max-pooling would be preferred; adapted from `reports/checkpoint_report.tex`, "Why global average pool at the head") followed by `Linear(96,128)`. The three 128-D descriptors are concatenated (**late fusion**, `v_{\text{final}} = [v_1;v_2;v_3] \in \mathbb{R}^{384}`) and passed through the classifier head:

$$z = W_2\,\operatorname{ReLU}(W_1 v_{\text{final}} + b_1) + b_2 \in \mathbb{R}^2, \qquad p(\text{contact}) = \operatorname{softmax}(z)_1 = \frac{e^{z_1}}{e^{z_0}+e^{z_1}},$$

matching `self.head = Sequential(Linear(384,64), ReLU(), Linear(64,2))` followed by `F.softmax(logits, dim=1)[0,1]` in `_run_x3d_inference`. Late fusion (concatenate-then-classify, rather than e.g. averaging per-camera probabilities) lets the classifier learn cross-camera relationships — e.g. "contact visible from camera 1 but not camera 2" is a different pattern than "contact visible from all three" — that a simple probability average would discard.

7.7 Implementation notes

- `resolution_behavior = "fixed"`: the spatial dimensions `(H,W)` of every `Conv3d` layer's receptive field interact with the sensor's native resolution through the two spatial-stride-2 blocks, so — as with `MobileNetSSDDetector` — one checkpoint per sensor profile is required (`sensor_profile.resolution` fixes `self._input_h`, `self._input_w` at construction).
- The rolling per-camera buffer (`deque(maxlen=T)`) implements the "fixed-step sampling" described in the module docstring: `predict()` is called once per incoming frame and always evaluates the network on the most recent `T` buffered frames, decoupling the network's temporal window from the capture frame rate.
- Training minimizes `nn.CrossEntropyLoss` (optionally class-weighted) via `Adam` — ordinary non-convex network training, as in §4 and §6; there is no KKT/duality argument for this model, only the standard (heuristic, not globally-guaranteed) convergence behavior of gradient descent on a non-convex loss surface.

References to reused report content

- §1.3, §2.1–§2.7 draw the six-feature description, the C-as-regularizer framing, the dual/kernel-trick narrative, the RBF-kernel interpretation, and the Platt-scaling paragraph from `reports/checkpoint_report.tex` ("FireSVM" section); the KKT stationarity/complementary-slackness/eligibility derivations (§2.4–§2.5) are original to this document, since the source report states the dual but does not derive KKT explicitly.

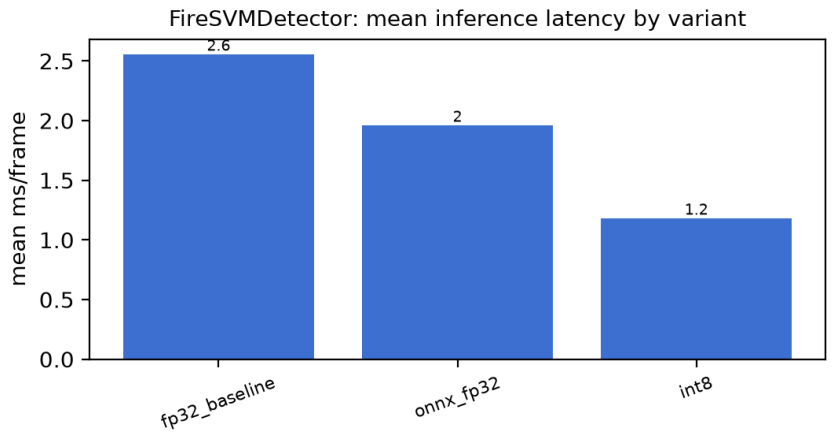
- §4.2, §4.5 (loss terms), §4.7 (NMS) draw the depthwise-separable-convolution cost argument, the CE+SmoothL1 loss narrative, and the NMS description from `reports/checkpoint_report.tex` ("MobileNet-SSD" section); the anchor-matching, hard-negative-mining, and encode/decode algebra (§4.3, §4.4, §4.5's offset derivation, §4.6) are derived directly from `mobilenet_ssd_anchors.py`/`mobilenet_ssd_model.py` since the source report does not derive them in closed form.
- §7.3–§7.6 draw the (2+1)D-factorization rationale, the CBAM-attention cost argument, and the global-average-pool justification from `reports/checkpoint_report.tex` ("Thermo-X3D T5v2" section); §7.5's global-normalization derivation and §7.2's 3-D-convolution generalization argument are original to this document, matching the actual `T=16`/global-mean-std implementation in `thermo_x3d.py` (distinct from the `T=5`/rolling-p25-background "T5v2" variant that report separately documents as a deployment configuration).
- `reports/fire_detection_report.tex` and `reports/human_detection_report.tex` were consulted for dataset/threshold context (§1.3, §3.6) but contain evaluation results rather than derivations, so their content is cited rather than reproduced.

Evaluation Results

All numbers below come from the exact same held-out waveshare_work test scenes for every detector and every variant (see scripts/eval_all_detectors.py:build_waveshare_test_split) -- fire, human, and contact detectors are all evaluated against the identical 3 sessions. "Mean ms" / "p95 ms" are single-frame inference latency (GPU for torch models by default, CPU for the two scikit-learn detectors and for every ONNX Runtime run on this machine -- its CUDA execution provider could not initialize here; see the ONNX section below).

FireSVMDetector

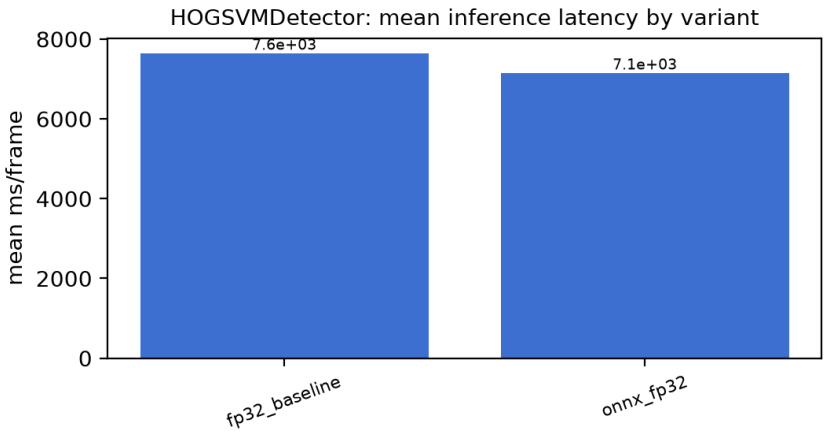
Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms
fp32_baseline	29.5%	29.5%	100.0%	45.6%	N/A	2.555	2.468
onnx_fp32	29.5%	29.5%	100.0%	45.6%	N/A	1.960	1.765
int8	29.5%	29.5%	100.0%	45.6%	N/A	1.181	0.655



HOGSVMDetector

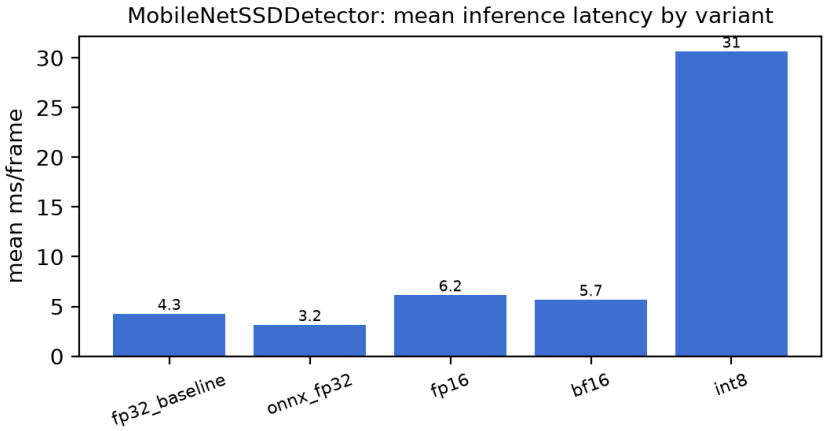
Timing caveat: HOGSVMDetector's brute-force multi-scale sliding-window search is inherently CPU-heavy (~2.8s/frame measured in isolation). Its evaluation pass here ran concurrently with the full-corpus training job on the same machine, so the mean/p95 ms below (~7-8s/frame) reflect CPU contention on top of that, not this detector's latency in isolation -- still the slowest detector in this report either way, just not apples-to-apples with the others' latency numbers, which ran without that contention.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms
fp32_baseline	97.0%	97.0%	100.0%	98.5%	57.2%	7641.506	8899.338
onnx_fp32	97.0%	97.0%	100.0%	98.5%	57.2%	7146.271	8348.422



MobileNetSSDDetector

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms
fp32_baseline	96.8%	97.0%	99.8%	98.4%	65.6%	4.281	5.341
onnx_fp32	96.8%	97.0%	99.8%	98.4%	65.6%	3.162	4.608
fp16	96.8%	97.0%	99.8%	98.4%	65.6%	6.158	9.234
bf16	96.8%	97.0%	99.8%	98.4%	65.6%	5.693	7.070
int8	96.8%	97.0%	99.8%	98.4%	65.7%	30.650	37.736

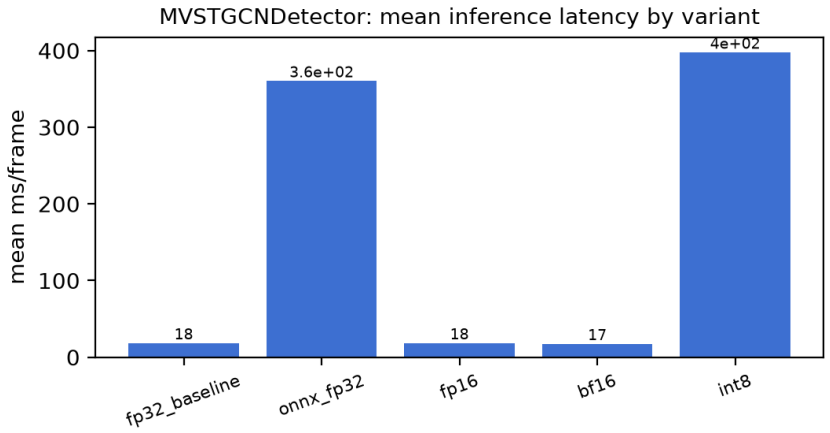


MVSTGCNDetector

Caveat (project owner): the homography used for this evaluation's actor positions is a synthetic per-camera-to-floor-plane transform (examples.utils.make_synthetic_homographies), used because no real on-site Hot-Point Calibration exists for any waveshare_work scene (see data/DATASET_NOTES.md). More fundamentally, the calibration approach available for real deployment fuses two cameras' views onto a third camera's image plane rather than rectifying each camera independently to the true floor plane -- so even a real calibration would not give the GCN

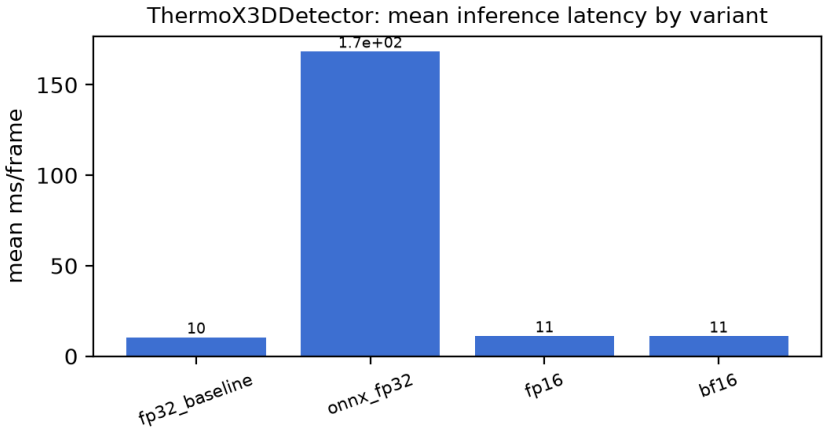
physically meaningful inter-actor distances, which is exactly what its epsilon_m/delta_m thresholds are calibrated against. Weak contact-detection precision/recall for MVSTGCNDetector below is therefore an expected consequence of the positional input signal, not evidence of insufficient training or model capacity.

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms
fp32_baseline	100.0%	0.0%	0.0%	0.0%	N/A	18.454	21.668
onnx_fp32	100.0%	0.0%	0.0%	0.0%	N/A	360.364	832.596
fp16	100.0%	0.0%	0.0%	0.0%	N/A	18.495	21.942
bf16	100.0%	0.0%	0.0%	0.0%	N/A	17.473	21.149
int8	100.0%	0.0%	0.0%	0.0%	N/A	397.623	512.672



ThermoX3DDetector

Variant	Acc	Prec	Rec	F1	IoU	Mean ms	p95 ms
fp32_baseline	100.0%	0.0%	0.0%	0.0%	N/A	10.497	15.693
onnx_fp32	100.0%	0.0%	0.0%	0.0%	N/A	168.493	251.479
fp16	100.0%	0.0%	0.0%	0.0%	N/A	11.166	17.215
bf16	100.0%	0.0%	0.0%	0.0%	N/A	11.059	17.862


ThermoX3DDetector: mean inference latency by variant
mean ms/frame
0
50
100
150
1.7e+02
10
11
11
fp32_baseline
onnx_fp32
fp16
bf16

ONNX Export Scope

Only the learned sub-component of each detector is traced into ONNX. Classical-CV pre/post-processing is deterministic NumPy/OpenCV code, not a neural-net or sklearn estimator, and isn't part of any framework's ONNX graph by construction -- see each row's scope note.

FireSVMDetector

StandardScaler + SVC(probability=True) only -- Otsu segmentation and blob feature extraction are classical CV code, not exported.

ONNX path: `C:\Users\guych\Desktop\Final Project\Weights\onnx\fire_svm_detector.onnx`

HOGSVMDetector

LinearSVC decision function only -- the HOG descriptor and multi-scale sliding-window pyramid are classical CV code, not exported.

ONNX path: `C:\Users\guych\Desktop\Final Project\Weights\onnx\hog_svm_detector.onnx`

MobileNetSSDDetector

Full backbone + SSD heads. Anchor decoding + NMS stay in Python (thermal_algorithms.human_detection.mobilenet_ssd_anchors) -- apply identically to torch and ONNX raw outputs for a fair comparison.

ONNX path: `C:\Users\guych\Desktop\Final Project\Weights\onnx\mobilenet_ssd_detector.onnx`

MVSTGCNDetector

Graph-conv body only, at a FIXED representative actor count (max_actors=5, mask-padded exactly as real inference does). Detection + Kalman tracking + homography projection + graph construction (thermal_algorithms.contact_detection.mv_stgcn / multi_view/*) stay in Python -- not neural-net ops.

ONNX path: `C:\Users\guych\Desktop\Final Project\Weights\onnx\mv_stgcn_detector.onnx`

ThermoX3DDetector

Full 3D-CNN body. Per-camera frame buffering + global normalization stay in Python.

ONNX path: `C:\Users\guych\Desktop\Final Project\Weights\onnx\thermo_x3d_detector.onnx`